



Herencia, Casting y Polimorfismo

Chuletario · DAM · Java OOP

Herencia

super

@Override

Casting

Polimorfismo

Concepto	Palabra clave	Idea
Herencia	extends	"ES UN"
Override	@Override	Redefinir método
Upcasting	automático	Hija → Padre
Downcasting	(Tipo)	Padre → Hija
instanceof	instanceof	Comprobar tipo
Polimorfismo	referencia padre	Múltiples formas

¿ES UN? → Herencia

Hija → Padre → Upcasting (auto)

Padre → Hija → Downcasting (cast + instanceof)



Herencia

extends · "ES UN" · Superclase → Subclase

Una clase hija **reutiliza y especializa** el comportamiento de la padre.
Solo se permite **herencia simple** en Java.

SINTAXIS

```
class Animal {
    protected String nombre;
    public Animal(String nombre) {
        this.nombre = nombre;
    }
    public void hacerSonido() {
        System.out.println("Sonido genérico");
    }
}

class Perro extends Animal {    // hereda
    public Perro(String n) {
        super(n);                // constructor padre
    }
    public void ladrar() {
        System.out.println("Guau!");
    }
}

Perro p = new Perro("Firulais");
p.hacerSonido(); // heredado del padre
p.ladrar();      // propio
```

QUÉ SE HEREDA

Modificador	¿Se hereda?	¿Accesible en hija?
private	✗ No	✗ No
protected	✓ Sí	✓ Sí
public	✓ Sí	✓ Sí
Constructores	✗ NO	Solo via super()

Orden de construcción: primero se construye el **padre**, luego la **hija**.
super() debe ser la **primera línea** del constructor hijo.

- ✗ class C extends A, B → **herencia múltiple prohibida**
- ✗ Los constructores **no se heredan nunca**



super · Object · Interfaces

Conectar con el padre · Raíz universal · Herencia múltiple

LOS 3 USOS DE SUPER

1. Constructor

super(nombre);
Llama al constructor padre. Primera línea siempre.

2. Atributo

super.nombre;
Accede al atributo del padre (si está oculto).

3. LLAMAR MÉTODO PADRE

```
class Perro extends Animal {
    @Override
    public void hacerSonido() {
        super.hacerSonido(); // → "Sonido genérico"
        System.out.println("Guau");
    }
}

// Salida: "Sonido genérico" → "Guau"
```

CLASE OBJECT · RAÍZ DE TODA CLASE JAVA

Toda clase hereda **implícitamente** de java.lang.Object
Por eso existen: toString() · equals() · hashCode()

Cadena de búsqueda de métodos:

1 Clase actual → 2 Padre → 3 Abuelo → ... → Object

HERENCIA MÚLTIPLE → INTERFACES

```
interface Volador { void volar(); }
interface Nadador { void nadar(); }

class Pato implements Volador, Nadador {
    public void volar() { System.out.println("Vuela"); }
    public void nadar() { System.out.println("Nada"); }
}
```

- ✓ Java solo permite **1 extends** (herencia simple)
- ✓ Con **implements** se pueden usar N interfaces



@Override - Sobrescritura

Redefinir el comportamiento heredado

La subclase **reemplaza** la implementación del método heredado.
Mismo nombre · mismos parámetros · mismo tipo de retorno.

EJEMPLO SIMPLE

```
class Animal {
    public void hacerSonido() {
        System.out.println("Sonido genérico");
    }
}
class Gato extends Animal {
    @Override
    public void hacerSonido() {
        System.out.println("Miau");
    }
}
```

VERRIDE EN JERARQUÍA REAL

```
public abstract class Factura {
    private double importe;
    public double getImporte() { return importe; }
    public abstract double calcularTotal(); // obliga
}
class FacturaConIva extends Factura {
    @Override
    public double calcularTotal() {
        return getImporte() * 1.21; // +21%
    }
}
class FacturaSinIva extends Factura {
    @Override
    public double calcularTotal() {
        return getImporte(); // sin IVA
    }
}
```

OCULTACIÓN DE ATRIBUTOS

```
class Padre { protected int valor = 10; }
class Hija extends Padre {
    protected int valor = 20;
    public void mostrar() {
        System.out.println(valor); // → 20
        System.out.println(super.valor); // → 10
    }
}
```

✓ CORRECTO

Mismo tipo de retorno
Mismos parámetros
Usar @Override

✗ INCORRECTO

Cambiar el tipo retorno
Cambiar parámetros
Reducir visibilidad



Casting entre Objetos

Upcasting · Downcasting · instanceof

Solo se puede hacer casting entre clases con **relación de herencia**.

↑ Upcasting

Hija → Padre
Automático · Siempre seguro
Animal a = new Perro();

↓ Downcasting

Padre → Hija
Explícito · Puede fallar
Perro p = (Perro) a;

UPCASTING – AUTOMÁTICO

```
Perro perro = new Perro();
Animal animal = perro; // automático ✓

animal.hacerSonido(); // OK – método de Animal
// animal.ladrrar(); ✗ no accesible (tipo Animal)
```

DOWNCASTING – EXPLÍCITO

```
Animal animal = new Perro(); // upcasting
Perro perro = (Perro) animal; // downcasting ✓
perro.ladrrar(); // OK
```

CLASSCASTEXCEPTION – ERROR TÍPICO

```
Animal animal = new Animal(); // objeto PURO
Perro perro = (Perro) animal; // ✗ EXCEPCIÓN
```

INSTANCEOF – OBLIGATORIO EN BUENAS PRÁCTICAS

```
if (animal instanceof Perro) {
    Perro p = (Perro) animal;
    p.ladrrar(); // seguro ✓
}

// Java 16+: pattern matching (más limpio)
if (animal instanceof Perro p) {
    p.ladrrar();
}
```

✓ El objeto sigue siendo su tipo **real** aunque la variable sea del padre

✓ Siempre comprueba con instanceof antes de hacer downcasting

✗ **Downcasting sin instanceof** → riesgo de **ClassCastException**

✗ **Intentar acceder a métodos de subclase sin casting**



Polimorfismo

Una referencia · múltiples comportamientos

Se ejecuta el método de la **clase real del objeto**, no el de la referencia.
El tipo de la variable *no* determina el comportamiento.

EJEMPLO CLÁSICO

```
Animal[] animales = new Animal[3];
animales[0] = new Perro("Firu");
animales[1] = new Gato("Misi");
animales[2] = new Animal("X");
```

```
for (Animal a : animales) {
    a.hacerSonido(); // Guau / Miau / Sonido genérico
}
```

CON INSTANCEOF + CASTING

```
for (Animal a : animales) {
    a.hacerSonido();
    if (a instanceof Perro p) p.ladrrar();
    if (a instanceof Gato g) g.mauullar();
}
```

MÉTODO POLIMÓRFICO

```
public static void imprimir(Animal a) {
    a.hacerSonido(); // ejecuta versión real
}
// Se puede pasar Perro, Gato, Animal...
```

IMPORTANTE

✓ Válido

Animal a = new Perro(); Padre ← Hija

✗ ERROR

Perro p = new Animal(); No se puede al revés

RESUMEN FINAL

Herencia = reutilización + especialización (extends)

super = constructor padre · atributo padre · método padre

@Override = mismo nombre/params/retorno · cambia lógica

Upcasting = automático · seguro · Hija→Padre

Downcasting = explícito · peligroso · usar instanceof

Polimorfismo = objeto real manda, no la referencia